Assignment 2 - Reduction

DUE Monday, October 19 at 11:59:59 PM

Overview

The objective of this assignment is to implement an optimized reduction kernel and analyze basic architectural performance properties.

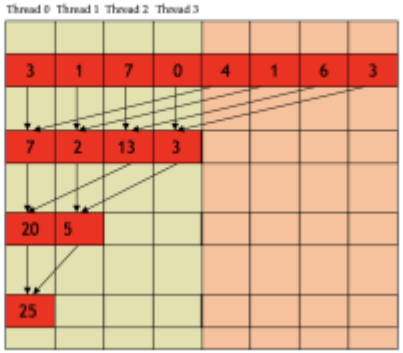
Naive Reduction

In the first step, you will implement a naive reduction kernel with unoptimized thread indexing (See Reduction slides for code snippets). Recall that we discussed two types of reduction in class: naive and optimized. The naive reduction kernel implementation suffers from significant warp divergence due to naive thread indexing.

Optimized Reduction

In the second step, you will implement an optimized reduction kernel with optimized thread indexing. Recall that we discussed two types of reduction in class: naive and optimized. The optimized version avoids a significant number of warp divergence. The goal is to have the thread indexing similar to the Optimized Reduction slides.

An Example of 4 threads



Github Classroom

1. For this lab, we will be using classroom50.org.
   Please accept the assignment by clicking the following link: https://classroom50.org/UCR-CSEE217/cs-ee217-fall-2026/assignments/reduction/accept.
   A private github repository will automatically be created with the starter code.
2. Clone the git repository. It contains:
   - `kernel.cu`: where you write your code
   - `main.cu`, `support.cu`, `support.h`, `Makefile`: provided; do not modify
   - `REPORT.md`, `plots/`: your written report (see Report)
3. The size of the input array is a command line argument. If none is given, 1 million is the default size.
   Note that you only have to accumulate the partial sums into an array, which is copied back to the host and verified to check the final answer.
   To ensure consistency when grading, do not change the srand seed value.
4. Complete the unoptimized and optimized reduction kernel by adding your code to `kernel.cu`. There should be no changes necessary for `main.cu`.
5. Build and verify the code works. `make` builds both versions: `./reduction` runs the naive kernel and `./reduction-optimized` runs the optimized kernel. Each prints `TEST PASSED` if every block's partial sum matches the CPU, or lists the blocks that are wrong.

```
make
./reduction              # naive, 1,000,000 elements
./reduction-optimized    # optimized, 1,000,000 elements
```

6. You will be profiling both versions of reduction to answer the questions in `REPORT.md`.

GPGPU-Sim analysis of warp divergence

> Please see the GPGPU-Sim setup guide.

We will now analyze the behavior of the naive and optimized Reduction kernel using GPGPU-Sim.
To aid in analyzing the microarchitectural properties of these programs, it may help to save the output of the GPGPU-Sim run into a file. You may save the output by redirecting the printouts to a file, e.g. `./reduction 10240 &> naive.out` and `./reduction-optimized 10240 &> optimized.out`. Remember to `make clean && make` inside the GPGPU-Sim image first, and again on Bender before running on the real GPU.

Since the focus of this lab is on performance and warp divergence, we will focus mainly on these statistics.
At the end of the simulation run, a lot of various statistics are printed out. Specifically, the following statistics are general performance statistics:

```
gpu_sim_cycle = 1672         // Number of cycles the last kernel took
gpu_sim_insn = 20884         // Number of instructions in the last kernel
gpu_ipc =      12.4904       // The IPC of the last kernel
gpu_tot_sim_cycle = 1672     // Number of cycles for the entire application (An application can consist of multip
```

```
gpu_tot_sim_insn = 20884     // Number of instructions for the entire application
gpu_tot_ipc =      12.4904   // The IPC of the entire application run
```

Another aspect we're concerned about is the warp divergence. In the output, there is a section of output that looks similar to this:

```
Warp Occupancy Distribution:
Stall:564       W0_Idle:3037    W0_Scoreboard:2257
W1:0    W2:0    W3:0    W4:9    W5:0    W6:0    W7:0    W8:0
W9:0    W10:0   W11:0   W12:0   W13:0   W14:0   W15:0   W16:310
W17:0   W18:0   W19:0   W20:0   W21:0   W22:0   W23:0   W24:0
W25:0   W26:0   W27:0   W28:1   W29:0   W30:0   W31:0   W32:512
```

This gives a distribution of the number of warps with a given number of active threads. For example, `W32` means that all 32 threads are active (i.e. no warp divergence). In this example, we also see a good number of warps that only have half of the threads active: `W16:310`.
The simulator defines WX (where X = 1 to 32) - The number of cycles when a warp with X active threads is scheduled into the pipeline.

Changing the block size

The thread block size is `BLOCK_SIZE` in `kernel.cu` (512 by default). Some questions ask you to try other block sizes: change the `#define BLOCK_SIZE` line, then rebuild with `make clean && make`. **Set it back to 512 before you submit.**

Automated tests

Every time you push to `main`, GitHub automatically builds your code and checks both kernels at several input sizes, including sizes that leave a partially filled last thread block. The program checks each block's partial sum against the CPU. It also runs `compute-sanitizer` to catch bugs that can still give the right answer by luck: out-of-bounds memory accesses, shared memory races (a missing `__syncthreads()`), and invalid `__syncthreads()` calls. Open the **Actions** tab of your repository to see the results. If a test fails, click on it: the summary explains the cause, and the log shows your program's output. You can run the same checks yourself, e.g.

```
./reduction 1025
compute-sanitizer --tool memcheck ./reduction 1025
compute-sanitizer --tool racecheck ./reduction-optimized 10240
```

Report

Answer the questions in `REPORT.md` in your repository. Expand "How to use this file" at the top of it for how to add plots, tables and math. When a question asks for a warp occupancy distribution, copy the `Warp Occupancy Distribution` section from the simulator output into a fenced code block, so its spacing is kept. If you make plots, save them as PNG files in `plots/` and embed them in `REPORT.md`.

Submission

1. Fill in your name and answers in `REPORT.md`, and add any plots to `plots/`.
2. Commit and push your completed naive and optimized reduction code (`kernel.cu`), `REPORT.md` and `plots/` to your GitHub repository. Open `REPORT.md` on GitHub to check that it displays correctly; that's the version we grade.
3. Check that the automated tests pass on your final commit.